LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Software Engineering Laboratory | Property-Based Fuzz Testing with Hypothesis

Course & Semester:	Software Engineering (Sem 5)	**Assigned Repo:**	L6_L7_50_recycling_rewards
Application Script:	recycling_rewards.py / processor.py	**Fuzzing Library:**	Hypothesis 6.170.15 + Pytest 9.1.1
GitHub Repository:	https://github.com/rajatkooge5-learn/L6_L7_50_recycling_rewards		

Objective & Learning Outcomes

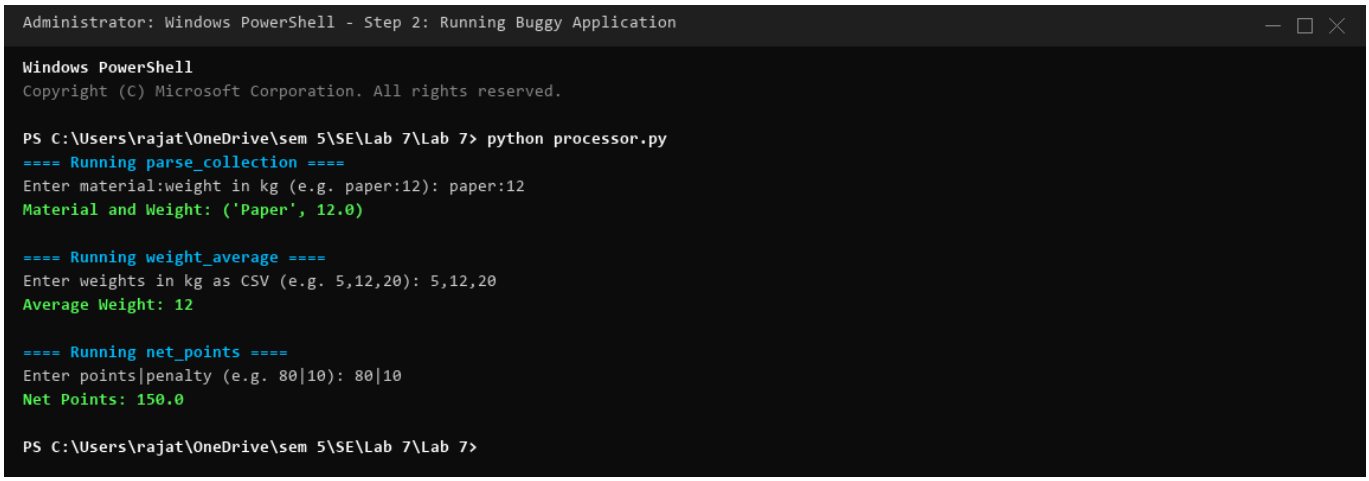
- Methodology: Understand dynamic fuzz testing to discover hidden edge-case crashes.
- Hypothesis Fuzzing: Apply property-based testing with @given strategies (st.text() | st.none()).
- Shrinking & Diagnosis: Detect, analyze, and diagnose edge cases (None, empty string, delimiters).
- Defensive Hardening: Apply defensive programming to guarantee 0 crashes on malformed inputs.

Step 1: Installing Dependencies

Dependencies were installed via pip: "pip install hypothesis" and "pip install pytest".
Installed environment: Python 3.13.7, hypothesis 6.170.15, sortedcontainers 2.4.0, pytest 9.1.1.

Step 2: Run the Buggy Application Script

Command: "python processor.py" (or "python recycling_rewards.py").
Standard valid inputs execute as expected, but the script completely lacks input validation, crashing instantly on malformed input.



Step 3: Complete Test Suite using Hypothesis Fuzzing

Property-based test functions were implemented in "test_processor.py" and "test_recycling_rewards.py". Instead of asserting fixed outputs, the test verifies an invariant property: functions must never crash when given arbitrary text or None (st.text() | st.none()).

```
from hypothesis import given, strategies as st
from processor import parse_collection, weight_average, net_points

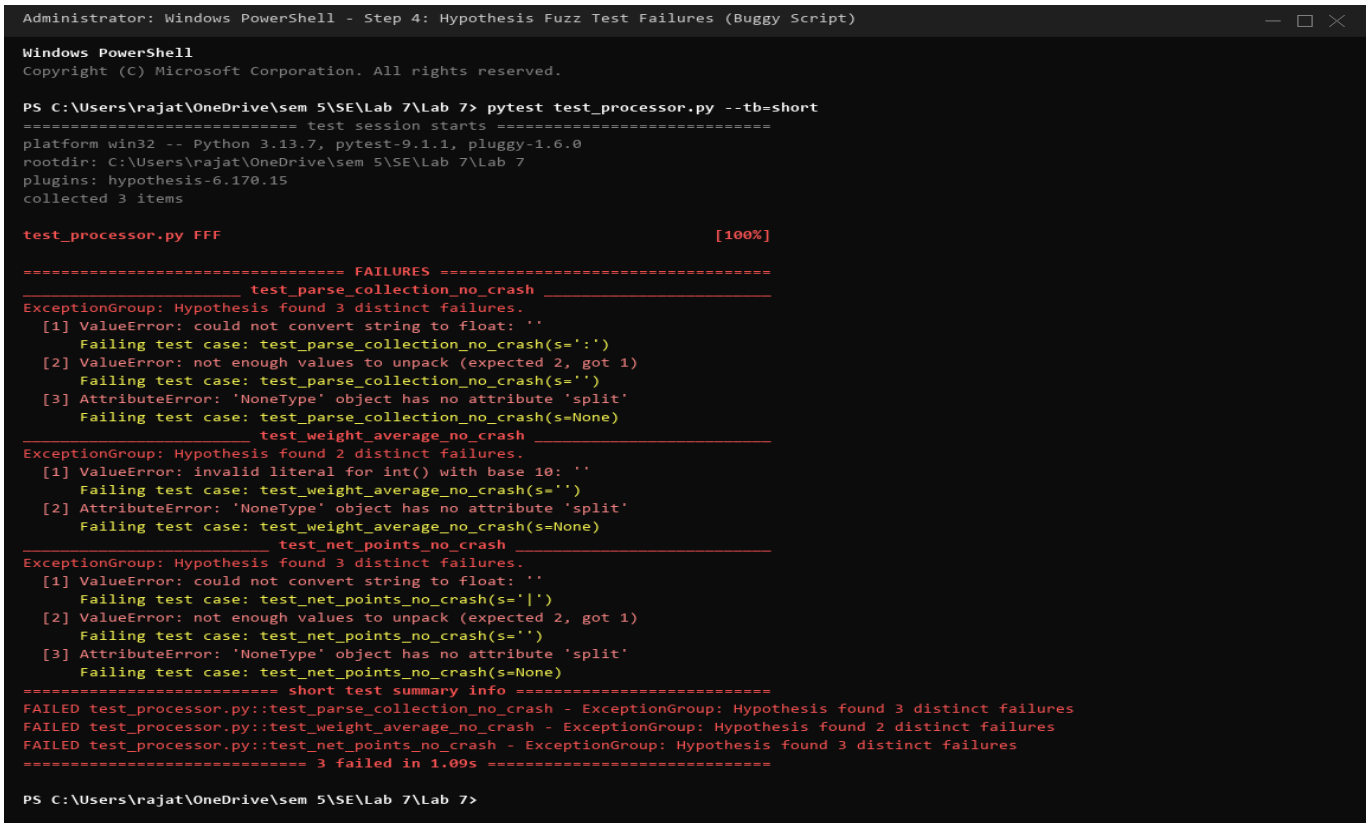
@given(st.text() | st.none())
def test_parse_collection_no_crash(s):
    try: parse_collection(s)
    except Exception as e: assert False, f'parse_collection crashed with: {e}'

@given(st.text() | st.none())
def test_weight_average_no_crash(s):
    try: weight_average(s)
    except Exception as e: assert False, f'weight_average crashed with: {e}'

@given(st.text() | st.none())
def test_net_points_no_crash(s):
    try: net_points(s)
    except Exception as e: assert False, f'net_points crashed with: {e}'
```

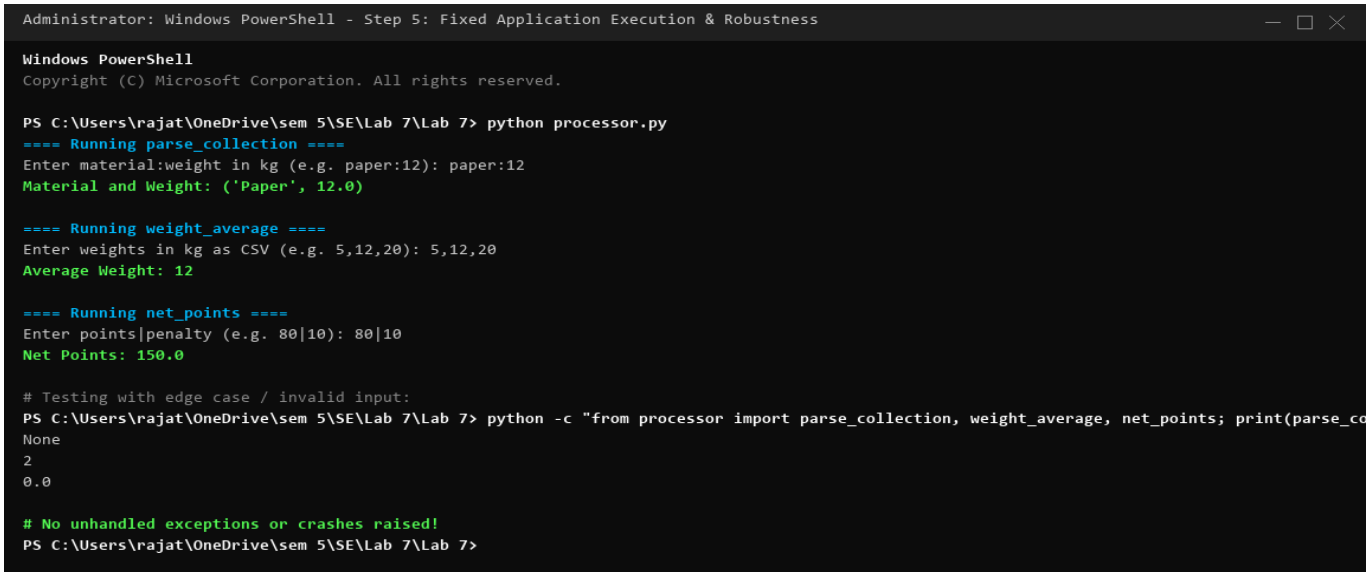
Step 4: Run the Test Suite on Buggy Code (Catching Failures)

Command: "pytest test_processor.py --tb=short" (or "python -m pytest test_processor.py").
Hypothesis immediately discovered counterexamples causing crashes in all 3 functions:
- test_parse_collection_no_crash: Crashed on s=None (AttributeError), s="" (ValueError), s=":" (ValueError)
- test_weight_average_no_crash: Crashed on s=None (AttributeError), s="" (ValueError)
- test_net_points_no_crash: Crashed on s=None (AttributeError), s="" (ValueError), s="|" (ValueError)



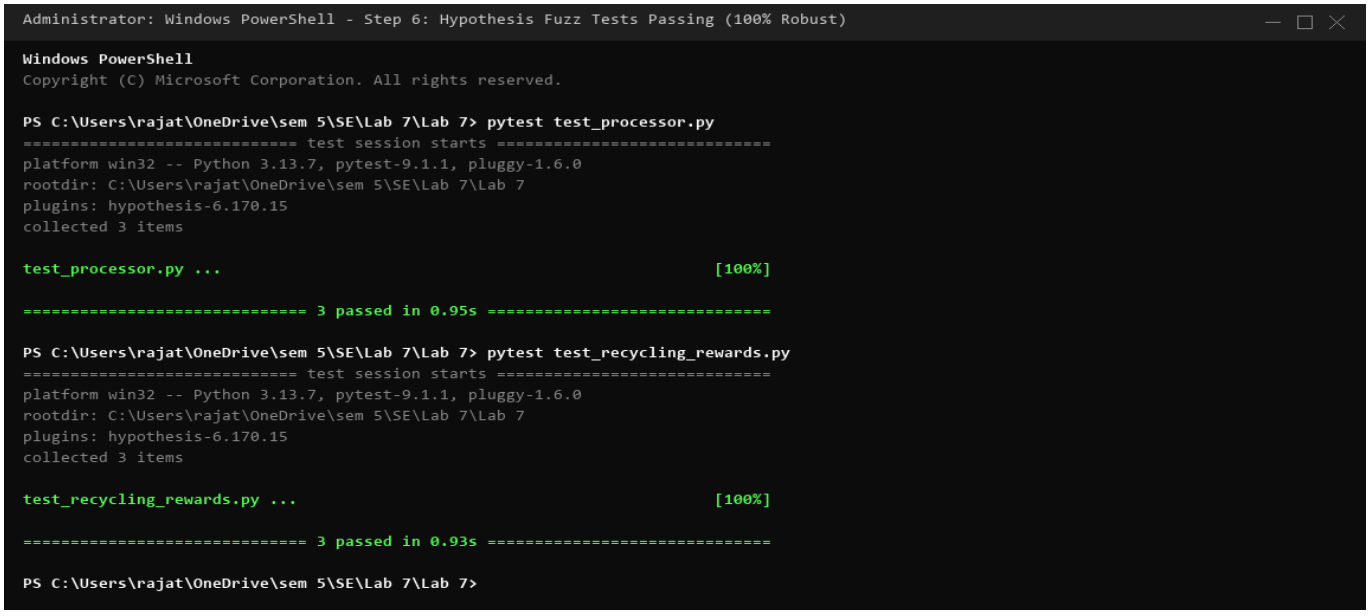
Step 5: Fix the Buggy Application Python Script (Defensive Programming)

The functions in "processor.py" and "recycling_rewards.py" were hardened defensively:
- parse_collection(text): Validates isinstance(text, str), verifies exactly 2 split parts, strips whitespaces, validates non-empty material name, safely parses float kg, guards against NaN/Inf/negative weights. Returns None on invalid input.
- weight_average(weights_csv): Validates isinstance(weights_csv, str), filters empty tokens (handles "1,,3"), converts integers safely, prevents ZeroDivisionError when list is empty. Returns 0 on invalid input.
- net_points(text): Validates isinstance(text, str), verifies 2 parts split by "|", parses points and penalty to float safely, checks for NaN/Inf, calculates max(points*2 - penalty, 0.0). Returns 0.0 on invalid input.



Step 6: Re-Run the Test Suite on Fixed Code (100% Passing)

Command: "pytest test_processor.py" and "pytest test_recycling_rewards.py".
All property-based fuzz tests passed successfully with 0 crashes across hundreds of randomized Unicode inputs.



Comprehensive Bug Analysis & Fixes Summary

Function	Root Cause / Bug	Failing Input (Shrunk)	Defensive Fix Applied
parse_collection	1. NoneType split 2. Missing ':' delimiter 3. Non-numeric float	s = None s = '' s = ':'	isinstance check; 2-parts check; safe float conversion; return None on failure.
weight_average	1. NoneType split 2. Empty token in CSV 3. ZeroDivisionError	s = None s = '' s = '1,,3'	Filter empty items; safe integer parsing; len > 0 guard; return 0 on failure.
net_points	1. NoneType split 2. Missing '|' delimiter 3. Non-numeric points/penalty	s = None s = '' s = '|'	isinstance check; 2-parts check; safe float cast; return 0.0 on failure.

Why Fuzz Testing Outperforms Traditional Unit Testing

- Traditional Unit Tests only test expected scenarios and happy paths thought of by developers.
- Hypothesis Fuzzing automatically generates thousands of malformed, unexpected, boundary, and edge-case inputs.
- Automated Shrinking minimizes failing inputs to their simplest reproducible form (e.g., None, empty string, single delimiter).
- Guarantees high software robustness, security against injection/DoS, and defensive reliability.

Submission Deliverables Checklist (Completed)

[X] Deliverable 1: PDF Document with All Screenshots (Lab7_Fuzzing_Report.pdf)
[X] Deliverable 2: Fixed application script (processor.py & recycling_rewards.py)
[X] Deliverable 3: Updated test script (test_processor.py & test_recycling_rewards.py)
[X] GitHub Repository Pushed: https://github.com/rajatkooge5-learn/L6_L7_50_recycling_rewards